Karlsruher Institut für Technologie

Institut für Programmstrukturen und Datenorganisation

Prof. Dr.-Ing. Gregor Snelting — gregor.snelting@kit.edu

Prof. Dr. Ralf H. Reussner — reussner@kit.edu

Klausur Programmierparadigmen

WS23/24, 28. März 2024, 08:00 – 10:00 Uhr

Zugelassene Hilfsmittel:
Papierbasierte Quellen (Vorlesungsfolien, Übungsblätter, eigene Aufzeichnungen, Bücher, ...)
Die Verwendung von elektronischen Geräten ist verboten.

Bearbeitungszeit: 120 min

Aufgabe	max. Punkte	err. Punkte
1	12	
2	22	
3	20	
4	10	
5	15	
6	12.5	
7	12.5	
8	16	
Σ	**120**	
$\lceil\Sigma\rceil$	**120**	

Jeder Punkt entspricht ca. 1 min Bearbeitungszeit. Es ist garantiert, dass die Klausur mit 60 Punkten bestanden ist, und dass 115 Punkte für die Note 1,0 ausreichen.

Name: _____

Matrikelnummer: _____

Studiengang: _____

Schreiben Sie Ihre Lösungen direkt in die Klausur. Beschriften Sie **alle verwendeten Blätter** mit Ihrem Namen und Ihrer Matrikelnummer. Trennen Sie die geklammerten Blätter **nicht** auf. Weitere Blätter erhalten Sie bei Bedarf. Wir behalten uns vor, Lösungen auf nicht von uns ausgeteilten Blättern nicht zu berücksichtigen.

Aufgabe 1 (Haskell, Listen) [12 Punkte]

(a) Geben Sie eine Funktion `transpose :: [[a]] -> [[a]]` an, welche eine Liste von Listen `xs :: [[a]]` transponiert. [5 Punkte]

Es soll z.B. gelten `transpose [[1,2],[3,4]] = [[1,3],[2,4]]`.

Hinweis: Sie dürfen annehmen, dass `xs` nur Listen gleicher Länge enthält.

(b) Definieren Sie die Funktion `intersperse :: a -> [a] -> [a]`. [4 Punkte]
Dabei soll `intersperse sep xs` den Separator `sep` zwischen alle Elemente der Liste `xs` einfügen.

Beispiel: `intersperse 'Y' "abc"== "aYbYc"`, `intersperse 'Y' "a"== "a"`.

(c) Definieren Sie die Funktion `nub :: [Int] -> [Int]`, die Duplikate aus der Eingabeliste entfernt. [3 Punkte]
Beispiel: `nub [3,1,3,5,5,7] == [3,1,5,7]`.

Aufgabe 2 (Haskell: Potenzreihen) [22 Punkte]

Eine (formale) Potenzreihe

$$\sum_{n=0}^{\infty} a_n \cdot x^n$$

kann in Haskell über die unendliche Liste ihrer Koeffizienten a_n, $n = 0, 1, \ldots$, dargestellt werden:

```
type Pow = [Double]
```

Ist `p :: Pow` eine solche Liste, so ist `p !! n` der Koeffizient a_n.

Hinweis: Es sind keine Analysis-Kenntnisse zur Lösung der Aufgabe erforderlich.

(a) Implementieren Sie die Funktion `fromCoeff :: (Int -> Double) -> Pow` zur Erzeugung einer Potenzreihe. Für `fromCoeff a` sind die Koeffizienten der Potenzreihe, beginnend mit `a 0`, direkt durch den Parameter `a :: Int -> Double` gegeben. [3 Punkte]

(b) Potenzreihen können addiert, multipliziert und differenziert werden. Implementieren Sie diese Operationen: [12 Punkte]

```
addP :: Pow -> Pow -> Pow
multP :: Pow -> Pow -> Pow
diffP :: Pow -> Pow
```

Verwenden Sie gegebenenfalls `sum` und `zipWith` sowie die Funktionen aus (a). Zur Erinnerung:

$$\left(\sum_{n=0}^{\infty} a_n \cdot x^n\right) + \left(\sum_{n=0}^{\infty} b_n \cdot x^n\right) = \sum_{n=0}^{\infty} (a_n + b_n) \cdot x^n$$

$$\left(\sum_{n=0}^{\infty} a_n \cdot x^n\right) \cdot \left(\sum_{n=0}^{\infty} b_n \cdot x^n\right) = \sum_{n=0}^{\infty} \left(\sum_{i=0}^{n} (a_i \cdot b_{n-i})\right) \cdot x^n$$

$$\left(\sum_{n=0}^{\infty} a_n \cdot x^n\right)' = \sum_{n=0}^{\infty} ((n+1) \cdot a_{n+1}) \cdot x^n$$

(c) Implementieren Sie die Funktion [7 Punkte]

```
evalP :: Pow -> Double -> Double -> Double
```

so dass `evalP p x eps` den Wert $\sum_{n=0}^{m-1} a_n \cdot x^n$ für das kleinste m, für das $|a_m \cdot x^m| < \epsilon$ gilt, berechnet.

Aufgabe 3 (Prolog: Labyrinth) [20 Punkte]

Im Folgenden soll ein Prolog-Programm zum Finden aller möglichen Wege durch ein Labyrinth entwickelt werden. Das Labyrinth sei in Felder unterteilt, die entweder *frei* oder *blockiert* sind.

In Prolog modellieren wir das Labyrinth über das Prädikat `frei(Feld)`, das genau dann erfüllbar ist, wenn das entsprechende `Feld` *frei* ist. Ein `Feld` ist ein Tupel `(X, Y)` aus X- und Y-Koordinate.

(a) Definieren Sie ein zweistelliges Prädikat [3 Punkte]

```
lengthof(L, Laenge)
```

das die Länge der übergebenen Liste `L` berechnet.

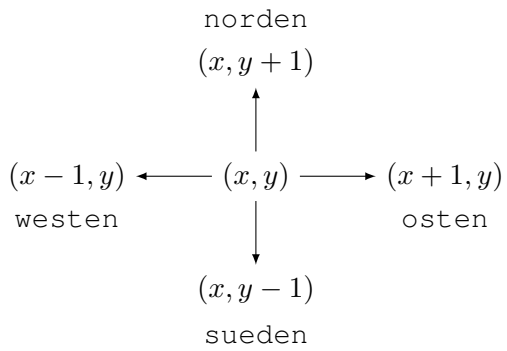
(b) Wir benötigen einen dreistelligen *Generator* [7 Punkte]

```
nachbar(Feld, Richtung, NachbarFeld)
```

der zu einem gegebenen `Feld` bei Reerfüllung alle *freien* Nachbarfelder `NachbarFeld` generiert. Schritte können nur in die Richtungen `norden`, `osten`, `sueden` und `westen` gemacht werden. Die Abbildung rechts gibt das verwendete Koordinatensystem an. Die Variable `Richtung` enthält eine Beschreibung, wie man vom Feld zum Nachbarfeld gelangt.

Beispiel:

```
? nachbar((1, 1), Richtung, NachbarFeld).
  Richtung = osten,
  NachbarFeld = (2, 1) ;
  Richtung = sueden,
  NachbarFeld = (1, 0) ;
  No
```



(c) Das Prädikat `wege(Schritte, MaxSchritte)` generiert alle möglichen Wege durch das Labyrinth, die höchstens `MaxSchritte` Schritte benötigen: [10 Punkte]

```
wege(Schritte, MaxSchritte) :-
        start(Start),
        ziel(Ziel),
        findeweg(Start, Ziel, MaxSchritte, [], Schritte).
```

Definieren Sie das hierzu benötigte fünfstellige Prädikat

```
findeweg(Start, Ziel, MaxSchritte, Besucht, Schritte)
```

welches für Startfeld `Start` und Zielfeld `Ziel` erfüllt ist, falls `Ziel` von `Start` aus durch eine Folge von höchstens `MaxSchritte` Schritten erreichbar ist. Weiterhin darf keines der in `Besucht` enthaltenen Felder nochmals betreten werden, um nicht im Kreis zu laufen. Die Liste `Schritte` sammelt die Richtungen der einzelnen Schritte.

Hinweis: Sie können die Prädikate `member` und `not` aus der Vorlesung, sowie `lengthof` aus Aufgabenteil (a) verwenden.

Aufgabe 4 (Church-Paare) [10 Punkte]

Aus der Vorlesung kennen Sie die Modellierung von Church-Zahlen und Church-Booleans. Auf ähnliche Weise kann man auch Paare von Elementen im λ-Kalkül darstellen. Der Paar-Konstruktor `pair` ist:

$$\texttt{pair} = \lambda \texttt{a}.\ \lambda \texttt{b}.\ \lambda \texttt{f}.\ \texttt{f a b}$$

Wichtige Funktionen auf Paaren sind die Destruktoren `fst` und `snd`, die jeweils das erste bzw. zweite Element aus dem Paar extrahieren. Die Definition von `fst` ist:

$$\texttt{snd} = \lambda \texttt{p}.\ \texttt{p}\ (\lambda \texttt{a}.\ \lambda \texttt{b}.\ \texttt{b})$$

(a) Definieren Sie `fst`. [1 Punkt]

(b) Zeigen Sie mit Beta-Reduktion unter Verwendung der Auswertungsstrategie Call-by-value: [4 Punkte]

$$\texttt{snd (pair a b)} \Rightarrow^* \texttt{b}$$

Fassen Sie dabei *keine* Reduktionsschritte zusammen. Als *Werte* im Sinne der Auswertungsstrategie zählen Lambda-Abstraktionen und die beiden freien Variablen `a` und `b`.

(c) In der Vorlesung wurde das Currying als Isomorphismus zwischen Funktionen auf Paaren und verschachtelten Funktionen vorgestellt. [5 Punkte]
Definieren Sie die Funktion `uncurry`, welche das Currying einer Funktion rückgängig macht und zeigen Sie, dass ihre Definition folgende Eigenschaft erfüllt:

$$\texttt{uncurry f (pair a b)} \Rightarrow^* \texttt{f a b}$$

Sie können hierzu die in der letzten Teilaufgabe gezeigte Reduktion und die entsprechende Eigenschaft von `fst` als gegeben annehmen.

Aufgabe 5 (Typinferenz) [15 Punkte]

Leiten sie den allgemeinsten Typ des folgenden Lambda-Terms mithilfe des Typinferenz-Algorithmus aus der Vorlesung her.

$$(\lambda x.x)((\lambda x.x)(\lambda z.(\lambda x.x)z))$$

Ihre Herleitung sollte wie folgt strukturiert sein:

(a) Stellen Sie den Typherleitungsbaum entsprechend den Regeln aus der Vorlesung auf.

(b) Geben Sie das sich ergebende Constraint-System an.

(c) Geben Sie eine Lösung des Constraint-Systems an. Eine Herleitung hierfür ist nicht erforderlich.

Aufgabe 6 (MPI) [12.5 Punkte]

Das folgende MPI-Programm soll einen Ring-Broadcast implementieren. Startend von Prozess 0 soll jeweils Prozess i die zu sendenden Daten erhalten und an Prozess i+1 weitersenden, bis alle Prozesse die Daten erhalten haben. Bei `N` gestarteten Prozessen soll Prozess `N-1` die Daten außerdem wieder an Prozess `0` senden. Das gegebene Programm enthält jedoch fünf Fehlerstellen. **Nennen** Sie für jeden der fünf Fehler die jeweiligen Programmzeile, in der er auftritt und **erklären** Sie kurz, was das Problem ist. **Erklären** Sie außerdem, wie Sie das Problem lösen können. Gehen Sie davon aus, dass das Programm immer mit mindestens zwei Prozessen ausgeführt wird sowie mit gültigen Parametern aufgerufen wird. Der Parameter `argv` enthält dabei die Eingabedaten.

Hinweis: Die Methode `atoi(const char *str)` konvertiert den String in `str` zu einem `int`. Sie können davon ausgehen, dass diese Methode nur mit gültigen Parametern aufgerufen wird.

```
1  #include <mpi.h>
2  #include <stdio.h>
3
4  int main(int argc, char** argv) {
5      int rank, size;
6      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
7      MPI_Comm_size(MPI_COMM_WORLD, &size);
8
9      int data;
10     // Ring broadcast
11     if (rank == 0) {
12         data = atoi(argv[1]);
13         MPI_Send(&data, 1, MPI_INT, (rank - 1), 0, MPI_COMM_WORLD);
14         MPI_Recv(&data, 1, MPI_INT, (rank - 1), 0, MPI_COMM_WORLD,
               MPI_STATUS_IGNORE);
15     } else {
16         MPI_Send(&data, 1, MPI_INT, (rank + 1), 0, MPI_COMM_WORLD);
17         MPI_Recv(&data, 1, MPI_INT, (rank - 1), 0, MPI_COMM_WORLD,
               MPI_STATUS_IGNORE);
18     }
19     MPI_Finalize();
20     return 0;
21 }
```

Aufgabe 7 (Java) [12.5 Punkte]

Der folgende Programmcode soll Teil eines Webshops sein, in dem Kunden Bestellungen (`Order`) aufgeben und stornieren können. Eine Bestellung kann dabei drei Zustände haben: offen (`Open`), ausgeliefert (`Delivered`) und storniert (`Cancelled`). Parallel zur Webshop-Funktionalität, die in mehreren Threads ausgeführt wird, soll nun eine Statistik erhoben werden, die den Durchschnitt der Lieferdauer aller offenen Bestellungen beinhaltet.

```
double calculateAverage(Object monitor, Iterator<Order> orders) { ... }
void write(double average) { ... }

void execute(ExecutorService executor, Object monitor,
        Iterator<Order> orders) {
```

```
}
```

(a) Vervollständigen Sie den oben aufgeführten Programmcode der Methode `execute`. Verwenden Sie den übergebenen `ExecutorService`, um ein `Future` zu erzeugen, in dem die Methode `calculateAverage` aufgerufen wird, die den Durchschnitt der erwarteten Lieferdauer aller offenen Bestellungen berechnet. Übergeben Sie den berechneten Wert dann an die Methode `write`. Fangen Sie möglicherweise auftretende `Exceptions` mit möglichst kleinen `try-catch`-Blöcken. [1.5 Punkte]

Hinweis: Es genügt, wenn Sie für `catch`-Blöcke den generischen `Exception`-Typ verwenden.

(b) Die Implementierung der Methode `calculateAverage` im nachfolgenden Java-Programmcode berücksichtigt nicht, dass andere Threads des Webshops gleichzeitig Objekte vom Typ `Order` hinzufügen und modifizieren können. Bereits angelegte `Order`-Objekte werden nicht wieder gelöscht. Vervollständigen Sie die Lücken im Programmcode der Methode, indem Sie einen geeigneten Synchronisierungsmechanismus einführen. Benutzen Sie dabei das übergebene `monitor`-Objekt als Monitor. Solange die Methoden des `Iterators` innerhalb der kritischen Bereichs des Monitors aufgerufen werden, kann der `Iterator` mit Änderungen an den zugrundeliegenden Daten umgehen. Um zu verhindern, dass der Webshop zu lange blockiert ist, soll nach der Bearbeitung jeder `Order` der Zugriff auf das gemeinsam genutzte `monitor`-Objekt wieder freigegeben und andere Threads darüber benachrichtigt werden. Die anderen Threads verhalten sich analog. Stellen Sie sicher, dass das Programm in jedem Fall terminiert und bei evtl. auftretenden Unterbrechungen (`InterruptedException`) ein valides Teilergebnis zurückgegeben wird. [4.5 Punkte]

Hinweis: Es werden nicht alle Lücken für eine korrekte Lösung benötigt und die Länge des erwarteten Programmcodes steht in keinem Verhältnis zur Größe der Lücken.

```
interface Order {
    OrderStatus getStatus();
    LocalDate getOrderDate();
    LocalDate getDeliveryDate();
}
enum OrderStatus { Open, Delivered, Cancelled }

int daysBetween(LocalDate from, LocalDate to) { ... }

double calculateAverage(Object monitor, Iterator<Order> orders) {
    int sum = 0, count = 0;
    [                                                    ]
    while (orders.hasNext()) {
        [                                                    ]
        Order order = orders.next();
        if (order.getStatus() == OrderStatus.Open) {
            sum += daysBetween(order.getDeliveryDate(), order.getOrderDate());
            count++;
        }
        [                                                    ]
    }
    [                                                    ]
    return (double)sum / count;
}
```

(c) In dieser Teilaufgabe sollen historische Bestelldaten, die als Liste `orders` vom Typ `List<Order>` vorliegen, betrachtet werden. Geben Sie dafür einen Ausdruck an, der mithilfe eines parallelen Streams das Maximum aller Lieferdauern von erfolgreich ausgelieferten Bestellungen (`OrderStatus.Delivered`) berechnet. [2.5 Punkte]

Hinweis: Verwenden Sie für die Berechnung der Lieferdauer die Methode `daysBetween` aus Teilaufgabe b).

```
orders.parallelStream()
```

(d) Im Folgenden ist eine nicht-parallelisierte Methode mit JML-Vertrag angegeben, die den Durchschnitt aller Lieferdauern von erfolgreich ausgelieferten Bestellungen (`OrderStatus.Delivered`) berechnet. Der JML-Vertrag für die Methode ist allerdings strikter als dafür benötigt. Ändern Sie die Vorbedingungen entsprechend ab, um unnötige Einschränkungen zu vermeiden. Stellen Sie dennoch sicher, dass bei der Berechnung des Durchschnitts nicht durch 0 geteilt wird und die betrachteten `Order`-Objekte gültig sind (Bestelldatum vor Lieferdatum). [4 Punkte]

```
/*@ requires !orders.isEmpty();
  @ requires (\forall int i;
            0 <= i && i < orders.size();
            order.getStatus() == OrderStatus.Delivered);
  @ requires (\forall int i;
            0 <= i && i < orders.size();
            order.getOrderDate().isBefore(order.getDeliveryDate()));
  @*/
double calculateAverageSequential(List<Order> orders) {
    int sum = 0, count = 0;
    for (Order order : orders) {
        if (order.getStatus() == OrderStatus.Delivered) {
            sum += daysBetween(
                order.getOrderDate(), order.getDeliveryDate());
            count++;
        }
    }
    return (double)sum / count;
}
```

Ihre Lösung:

Aufgabe 8 (Compiler) [16 Punkte]

Gegeben sei die folgende Grammatik für Prolog-Fakten mit dem Startsymbol *Fact*:

$$
\begin{array}{rcl}
\mathit{Fact} & \rightarrow & \mathit{Term}\ \texttt{.} \\
\mathit{Term} & \rightarrow & \texttt{atom}\ \mathit{Func} \mid \texttt{number} \mid \texttt{variable} \\
\mathit{Func} & \rightarrow & \varepsilon \mid \texttt{(}\ \mathit{Termlist}\ \texttt{)} \\
\mathit{Termlist} & \rightarrow & \mathit{Term}\ \mathit{Termlist}' \\
\mathit{Termlist}' & \rightarrow & \varepsilon \mid \texttt{,}\ \mathit{Term}\ \mathit{Termlist}'
\end{array}
$$

(a) Geben Sie die First_1- und Follow_1-Mengen für die Nichtterminale *Fact* und *Term* an. [4 Punkte]

(b) Geben Sie für jedes Nichtterminal der Grammatik einen rekursiven Abstiegsparser nach dem Schema der Vorlesung an. Brechen Sie bei Parserfehlern mit der globalen `error()`-Funktion ab. [12 Punkte]

Hinweis: Sie können davon ausgehen, dass die Lexer-Schnittstelle wie folgt aussieht:

- Die Token sind Werte eines `enum` namens **`Token`** und heißen **`ATOM`**, **`NUMBER`**, **`VAR`**, **`DOT`**, **`COMMA`**, **`LP`**, **`RP`** und **`EOF`** für `atom`, `number`, `variable`, `.`, `,`, `(`, `)`, sowie das Ende der Eingabe. Token können bspw. mit `==` verglichen werden.
- `lexer.current` gibt nichtkonsumierenden Zugriff auf das aktuelle Token.
- `lexer.lex()` konsumiert das aktuelle Token.
- Die folgende Funktion ist verfügbar:

```
void expect(Token tok) {
    if (lexer.current != tok) error();
    lexer.lex();
}
```